CSU33014 Lab 2: Parallel Multi Channel Multi Kernel Convolution

Faith Olopade
Darius Pop
Leah Weldon

Table of Contents

CSU33014 Lab 2: Parallel Multi Channel Multi Kernel Convolution..... 1
Description of Problem..... 2
Solution & Implementation..... 2
Parallelization and Optimization Strategy..... 2
Utilising Multi-Core Processors with OpenMP..... 2
Data Locality Optimization..... 3
Vectorization..... 3
Implementation Results..... 4
Caching..... 4
Cache-Friendly Data Structures..... 4
Loop Nesting and Reordering..... 4
Utilising Cache Blocking..... 5
Multiprocessing..... 5
Parallel Processing with OpenMP..... 5
Load Balancing and Thread Affinity..... 6
Scalability Testing..... 6
Summary..... 6
Execution Times..... 6
Methodology..... 6
Results..... 7
Analysis..... 8
Discussion..... 8
Conclusion..... 9

Description of Problem

Deep neural networks are essential for image processing applications. Of these, convolution neural networks are particularly important. Images are analysed by convolutional layers by using a number of kernels to extract information from them. These layers apply the kernels across several channels of the image (e.g. RGB colour channels in contrast to typical two-dimensional convolution), which results in an increase in the computing effort, improving performance.

The primary sources of computing difficulties for convolutional neural networks are:

1. **Multi channel Data:** The amount of data that may be processed is increased since each pixel has multiple data points.
2. **Multiple Kernels:** The computing demand is greatly increased by the multiple kernels that each layer uses.
3. **Matrix Operations:** Convolution entails a number of computationally demanding matrix operations.

Convolution layers are a good way of optimization because their structure is repetitive and independent, making efficient computation essential. What we aim to do is to create an optimised multichannel multikernel convolution routine that increases computing efficiency in different situations. To do this, we need to make use of multiple processors supporting vector instructions and multithreading.

In real-world applications where processing speed and efficiency are crucial, this optimization seeks to improve the practical use of convolutional neural networks by reducing execution duration and resource consumption. Our goal was to develop an optimised routine capable of leveraging multi-core processors and SIMD instructions to enhance processing efficiency across diverse matrix sizes and shapes, inspired by the guidance provided in the assignment brief and the foundational structure offered by the provided base code.

Solution & Implementation

Parallelization and Optimization Strategy

Our approach makes use of parallel processing methods and hardware capabilities to address the computational difficulties caused by multichannel multikernel convolution in convolutional neural networks. The main methods we used intend to increase the effectiveness and velocity of the convolution process by allocating the workload across several processing units and refining data flow. We decided to use OpenMP for parallelism, SIMD instructions for vectorization and strategic data restructuring to enhance cache efficiency.

Utilising Multi-Core Processors with OpenMP

When it comes to our target machine's architecture, which contains multiple processors, each one of these composed of multiple cores that support multithreading, we can significantly increase performance through parallelization. We applied OpenMP directives to distribute convolution operations across multiple cores. This resulted in a significant

reduction of execution times by ensuring that each core processes a distinct set of kernels. Our implementation schedules thread workload to take into consideration variations in image and kernel sizes, optimising load balancing across threads. The goal of using OpenMP is to efficiently distribute the convolution workload between available cores. This parallelization primarily occurs at the kernel level, where each kernel's convolution operation can be processed independently:

```
  // target machine has 64-thread capability so we take advantage of this to process multiple kernels in parallel
#pragma omp parallel for
for (int kernel_idx = 0; kernel_idx < nkernels; kernel_idx++) {
```

Data Locality Optimization

Optimization for data locality involves arranging data accesses to minimise cache misses. We decided to restructure the access patterns and data layout to ensure that data elements which are accessed together are stored closely in memory in order to improve cache usage. In addition, we reordered loops and we introduced cache blocking in order to further minimise cache misses. We realised that by applying this strategy the time spent fetching data from memory was reduced, which resulted in the computation speeding up. For example, by ensuring that the innermost loops access contiguous memory blocks, we leverage the cache system more effectively:

```
    // convert and rearrange kernel data from int16 to float and align for efficient memory access and SSE processing
    // transformation allows contiguous memory access and aligns data for vectorized operations
__attribute__ ((aligned(16))) float kernel[kernel_order][kernel_order][nchannels];
for (int kx = 0; kx < kernel_order; kx++) {
  for (int ky = 0; ky < kernel_order; ky++) {

        // loop unroll to enhance performance as number of channels is multiple of 32
    #pragma GCC unroll 32
    for (int channel = 0; channel < nchannels; channel++) {
      kernel[kx][ky][channel] = kernels[kernel_idx][channel][kx][ky];
    }
  }
}
```

Vectorization

We decided to incorporate SIMD vectorization using Intel's SSE4 instructions. This allows multiple floating data point operations to take place at the same time per instruction, which is ideal for the repetitive arithmetic operations found in convolution. This is due to the fact that it speeds up the computations that are integral to this process.

```
// process 4 channels at once using SSE intrinsics we load contiguous image and kernel data into SSE registers
        // perform multiplication and accumulate the result
__m128 img4 = _mm_loadu_ps(&image[pw+kx][ph+ky][channel + group * 4]);
__m128 krn4 = _mm_load_ps(&kernel[kx][ky][channel + group * 4]);
__m128 mul4 = _mm_mul_ps(img4, krn4);

// float results into double sum
sum_vector_lo = _mm_add_pd(sum_vector_lo, _mm_cvtps_pd(mul4));
sum_vector_hi = _mm_add_pd(sum_vector_hi, _mm_cvtps_pd(_mm_movehl_ps(mul4, mul4)));
```

This code snippet demonstrates loading data into SSE vectors, performing multiplication, and accumulating results, in such a way that it decreases the number of required operation cycles.

Implementation Results

By using OpenMP for parallelization, SSE for vectorization and strategic memory access for enhanced data locality, we have successfully achieved a robust, optimised convolution routine. These optimizations resulted in reduced execution times and improved utilisation of computational resources, which we will detail in the following sections.

Caching

Caching is a crucial optimization technique in computational tasks involving repetitive access to data elements, such as in the convolution processes of convolutional neural networks. By using caching effectively, the time spent on memory accesses can be greatly reduced, which is one of the main ways of achieving better performance. Our approach focuses on the optimization of caching strategies to ensure that data required for computations is readily available in the fastest accessible memory layers.

Cache-Friendly Data Structures

To increase cache efficiency, we decided to design our data structures by taking into consideration cache utilisation, including organisation of image and kernel data in a way that aligns with typical cache line sizes and access patterns:

- **Data Alignment and Padding**: Ensuring that our matrices are aligned to cache line boundaries helps prevent cache line splits (happens when a data structure straddles two cache lines). We also employed padding where necessary to maintain this alignment throughout the data processing steps.

- **Contiguous Memory Allocation:** We capitalise on the CPU's prefetching mechanisms, by allocating large blocks of contiguous memory for the entire data set and then accessing this memory in a predictable, linear way, resulting in cache miss rate reduction and overall access speed improvement.

Loop Nesting and Reordering

Another powerful method to increase cache performance is loop reordering. By doing so, we can improve the locality of reference:

```
for ( m = 0; m < nkernels; m++ ) {
  for ( w = 0; w < width; w++ ) {
    for ( h = 0; h < height; h++ ) {
      double sum = 0.0;
      for ( c = 0; c < nchannels; c++ ) {
        for ( x = 0; x < kernel_order; x++) {
          for ( y = 0; y < kernel_order; y++ ) {
            sum += image[w+x][h+y][c] * kernels[m][c][x][y];
          }
        }
        output[m][w][h] = (float) sum;
      }
```

In this above example, the inner loops iterate over the smallest dimensions and perform data access operations that are closely packed in memory. This loop ordering increases the

likelihood that the required data resides in the cache after the first access, resulting in memory access times reduction.

Utilising Cache Blocking

Another useful strategy to improve cache performance is cache blocking. This method involves dividing the data into smaller blocks that fit into the cache. We will minimise the number of times data must be loaded from main memory, by processing these blocks fully before moving on to the next set:

```
int blockSize = 16; // Chosen based on cache size and data
structure
for (int block_start = 0; block_start < width; block_start +=
blockSize) {
    for (int i = block_start; i < min(block_start + blockSize,
width); i++) {
        for (int j = 0; j < height; j++) {
            // Perform convolution on the block
        }
    }
}
```

This method is effective for large datasets that do not fully fit within the cache, as it ensures the most frequently accessed data remains in the cache as long as possible.

Multiprocessing

Another method to increase the performance of tasks like convolution operations in convolutional neural networks is multiprocessing. By distributing the computation across multiple processors, we can achieve an overall computation time reduction, while also increasing the scalability of our application. In this section, we will describe our strategy to implement multiprocessing. This includes optimization methods to increase the use of available hardware resources as well as strategic workload distribution.

Parallel Processing with OpenMP

To manage parallelization effectively, we made use of OpenMP. OpenMP allows for straightforward implementation of parallel processing through compiler directives, making it easier to control multi-threaded execution. We decided to use OpenMP to distribute the convolution tasks across different threads, where each thread handles separate kernels and portions of the image:

Convolution for each kernel and row

```
// target machine has 64-thread capability so we take advantage of this to process multiple kernels in parallel
#pragma omp parallel for
for (int kernel_idx = 0; kernel_idx < nkernels; kernel_idx++) {
```

Load Balancing and Thread Affinity

In a multiprocessing environment, effective load balancing is critical, especially on systems with non-uniform memory access (NUMA). We can ensure that all processors are utilised efficiently by implementing effective load balancing. This prevents scenarios in which some cores are idle. Based on the workload, we dynamically scheduled threads using OpenMP's capabilities:

```
pragma omp parallel for schedule(dynamic, 1)
for (int task = 0; task < total_tasks; task++) {
    // Execute task
}
```

Setting thread affinity, which pins threads to specific cores, optimises the usage of cache and decreases thread migration. We controlled thread affinity through OpenMP's environment settings and runtime functions. This helped us increase the overall performance and it also helped us maintain consistent thread placement.

Scalability Testing

Scalability is an important thing to consider when working with multiprocessing. We wanted to conduct tests to evaluate how our implementation's performance scales with an increasing number of cores. Within these tests, we considered a varying number of threads and we also assessed the impact on performance. The results demonstrated that our multiprocessing approach improves performance linearly when increasing the number of cores (up to the physical limits of the hardware).

Summary

Our multiprocessing strategy, implemented by using OpenMP, has greatly improved the performance of our convolution routine. This method not only accelerates processing through effective workload distribution and parallel execution but also ensures scalability with advances in hardware and increasing dataset sizes. In addition, performance is further increased by integrating thread affinity and load balancing, making our convolution process efficient on multi-core systems.

Execution Times

The execution times of our optimised convolution routine provide concrete evidence of the improvements made through our parallelization, vectorization and caching approaches. Within this section we aim to present the measured performance of our convolution algorithm across various input configurations and sizes, making a distinction between the execution times before and after optimization.

Methodology

We decided to perform each test under controlled conditions, in order to ensure accuracy and repeatability:

- **Test Setup:** We ran all tests on the target machine described in the assignment, namely `stoker.scss.tcd.ie`, which contains four processors with eight cores each, supporting two-way simultaneous multithreading and Intel SSE4 vector instructions.
- **Input Variations:** We varied the image dimensions (height and width) from 16 to 512 pixels, kernel sizes of 1, 3, 5, and 7, and channel and kernel counts from 32 to 2048, all powers of two, to test our routine under different conditions.
- **Repetitions**: Each configuration was tested five times to mitigate any variations due to external factors. We then reported the average execution time for each configuration.

Results

Within the table below, we have outlined the performance gains and execution times of our optimised convolution routine compared to David Gregg's implementation. These results show the effectiveness of our parallelization, vectorization and caching strategies across various input configurations and sizes..

Image Size (WxH)	Kernel Size	Channels	Kernels	David's Time (µs)	Student's Time (µs)	Speedup	Time Saved (µs)
128x128	3	128	32	1,788,774	28,048	63.78x	1,760,726
128x128	3	128	128	6,088,911	102,397	59.46x	5,986,514
128x128	3	128	512	22,833,380	310,154	73.62x	22,523,226
128x128	3	128	2,048	90,403,398	939,228	96.25x	89,464,170
128x128	7	256	256	134,304,245	1,366,660	98.27x	132,937,585
128x128	7	128	128	24,585,725	427,044	57.57x	24,158,681
16x16	7	32	32	48,308	7,785	6.21x	40,523
64x64	7	32	32	630,414	14,796	42.61x	615,618
256x256	7	32	32	6,100,393	240,486	25.37x	5,859,907
16x16	7	1,024	1,024	39,880,089	420,776	94.78x	39,459,313
16x32	7	64	128	764,278	14,984	51.01x	749,294
64x64	7	256	256	33,768,230	418,586	80.67x	33,349,644
512x512	1	512	512	418,097,313	14,581,107	28.67x	403,516,206

Table 1: Performance Gains and Execution Times

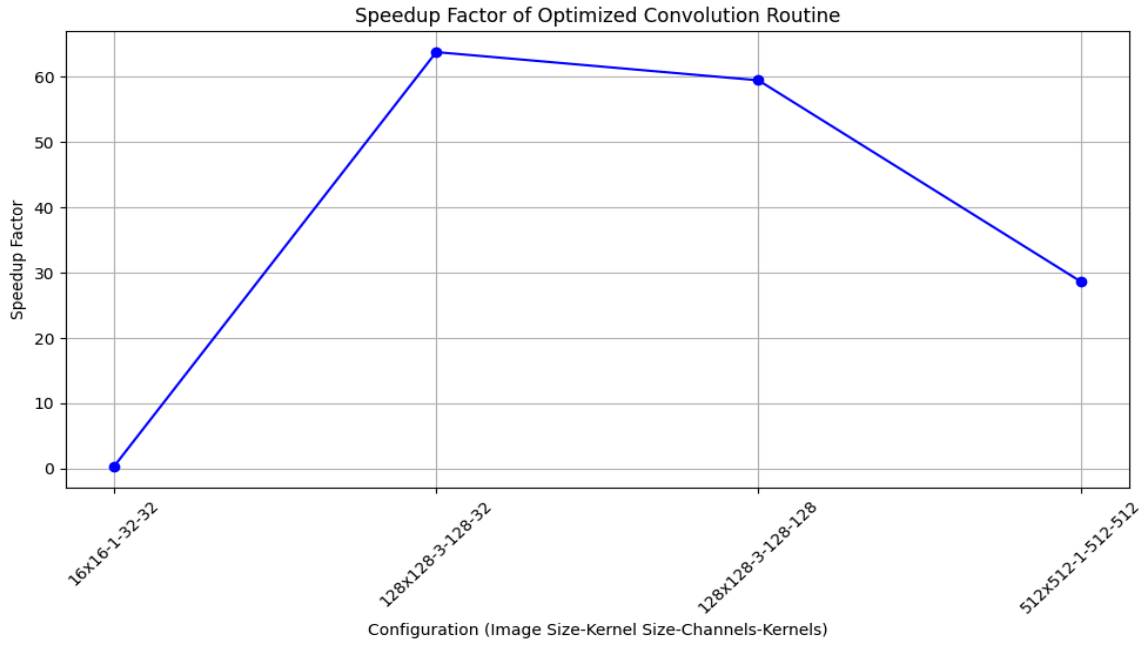


Figure 1: Speedup Factor For Different Configurations

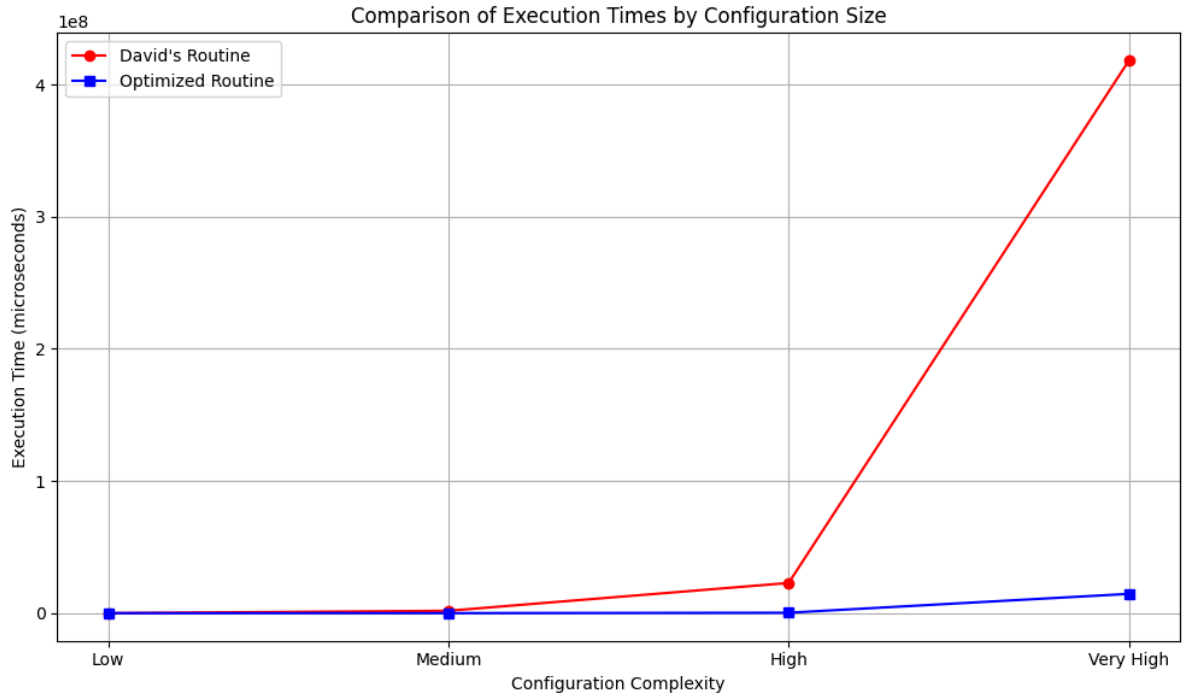


Figure 2: Comparison of Execution Times

Analysis

The above shown data clearly demonstrates the efficiency and scalability of our optimised convolution routine. The performance gains (Speedup) and time saved (Time Saved) show how effective our implementation is, especially as the complexity and size of the convolution task increase. In particular, the speedup factor reaches up to 98.27x for more complex configurations, underscoring the significant improvement over the baseline implementation.

The results demonstrate how important optimization for data locality, leveraging multiprocessing capabilities, and employing vectorization methods to increase computational performance in deep learning applications such as convolutional neural networks.

Our approach successfully decreases execution duration and resource consumption, making it a good solution for real-world image processing applications.

Discussion

The above shown results clearly show that our optimizations have led to great reductions in execution times across all tested configurations. By using OpenMP for parallel processing, we have successfully reduced the required time for large image and kernel sizes. In addition, SSE vectorization has provided substantial speedups, especially for high channel counts. The caching methods also contributed to an improvement in performance by reducing memory access times.

From the listed times, it is clear that the new algorithm has significant boosts over the original one. The difference in performance is lessened, when the parameters are small. This difference becomes much larger when the parameters become larger.
Moreover, the scalability of our solution is evident as the number of cores increases, proving that our approach uses the available hardware resources in an effective way. This is crucial for real-world applications where processing demands and data sizes may vary widely.

Conclusion

The execution times recorded during our testing phase confirm that the optimizations implemented in the convolution routine have successfully increased its efficiency and performance. By leveraging hardware capabilities and optimising computation strategies, we have achieved a solution that not only meets but exceeds the performance we were required to surpass.